

DoomOS VMware Integration & Deployment Guide

This guide explains how to construct, test, and deploy **DoomOS (KDE Plasma 6 Edition)** using **VMware Fusion** (macOS) and **VMware Workstation** (Windows & Linux) **without using Docker**.

Overview: Two Docker-Free Workflows

Workflow	Where Build Happens	How It Works	Best For
Option A (Fastest & Automated)	GitHub Actions Cloud	GitHub Actions builds the ISO on native x86_64 cloud runners, splits it into `part01` & `part02`, and uploads to GitHub Releases. You download chunks, run `./combine.sh`, and boot in VMware.	macOS Apple Silicon, low-spec laptops, or zero local setup.
Option B (Self-Hosted Local)	Inside VMware Arch VM	Boot an Arch Linux VM in VMware, share this folder via `/mnt/hgfs`, and run `sudo ./build-in-vmware.sh`. Builds 100% natively in Linux with zero Docker.	Developers who want to compile entirely locally on a Linux VM.

Option A: GitHub Cloud Build $\rightarrow$ VMware Boot (Recommended)

Step 1: Push Repository to GitHub

Once pushed to `https://github.com/himanshubhagat651/doomOS`:

- The GitHub Actions workflow ([`.github/workflows/build-and-release.yml`](.github/workflows/build-and-release.yml)) will build the complete DoomOS ISO on native 8-core x86_64 servers.
- It generates:
 - `doomos-plasma-x86\_64.iso.part01` (~2.0 GB)

- `doomos-plasma-x86\_64.iso.part02` (~1.0 GB)
- `doomos-plasma-x86\_64.iso.sha256`
- `combine.sh`

Step 2: Download & Recombine

1. Download `part01`, `part02`, and `combine.sh` into your folder.
2. Run the reassembly script:

```
```bash
chmod +x combine.sh
./combine.sh
```
```

3. The script concatenates the chunks and verifies the cryptographic SHA256 checksum automatically.

Step 3: Launch in VMware

Run the automated VMware runner:

```
chmod +x test-vmware.sh
./test-vmware.sh
```

This generates the optimized `DoomOS.vmwarevm` bundle with:

- **Firmware:** Modern UEFI (`firmware = "efi"`)
- **CPU & Memory:** 4 vCPUs, 4096 MB RAM
- **Display:** 3D SVGA hardware acceleration enabled (`mks.enable3d = "TRUE"`)
- **Disk:** High-performance NVMe virtual disk + SATA CD-ROM with DoomOS ISO.

Option B: Building Natively Inside a VMware VM (No Docker)

If you prefer to compile the ISO entirely on your local machine using VMware:

Step 1: Create an Arch Linux VM in VMware

1. In VMware Fusion / Workstation, create a new VM using a standard Arch Linux base ISO.
2. Allocate **4 CPU cores** and **6 GB RAM**.
3. Enable **Shared Folders** in VM Settings and share this ``dr doom`` project directory.

Step 2: Run the Native Build Script

Boot into your Arch Linux VM, mount the shared folder, and run:

```
# Inside the VMware Arch Linux VM:
cd /mnt/hgfs/dr\ doom/
sudo chmod +x build-in-vmware.sh
sudo ./build-in-vmware.sh
```

What ``build-in-vmware.sh`` Does:

1. Installs native ISO mastering tools (``archiso``, ``squashfs-tools``, ``xorriso``, ``grub``, ``zstd``) directly via ``pacman``.
2. Compiles and compresses the root filesystem with maximum Zstandard ratio (``zstd -Xcompression-level 19 -b 1M``).
3. Masters the bootable hybrid UEFI/BIOS ISO into ``doomos-builder/output/doomos-plasma-x86_64.iso``.
4. Computes the SHA256 checksum.

🌟 Native VMware Features Built into DoomOS

DoomOS is pre-configured with first-class VMware guest integration:

1. **Dynamic Screen Resizing:** Plasma 6 automatically adapts its Wayland desktop resolution to match your VMware window.
2. **Bidirectional Clipboard:** Seamless copy and paste between host and guest.
3. **VMware Shared Folders (``vmhgfs``):** Access host files directly from ``/mnt/hgfs``.
4. **Hardware 3D Acceleration:** Uses the native Linux ``vmwgfx`` DRM kernel driver with Mesa SVGA 3D.

5. **Auto-started Services:** ``vmtoolsd.service`` and ``vmware-vmblock-fuse.service`` are enabled by default.